OS & SP

Name : D. Teja Sadwika **Roll No :** 2520030536

SKILL – 06

1.Skill6.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100

/*
 * Parse command and handle escaped characters.
 *
```

```
 * Example:
 * echo hello world
 * argv[0] = echo
 * argv[1] = hello
 * argv[2] = world
 *
 * echo hello\ world
 * argv[0] = echo
 * argv[1] = hello world
 */

int parse_command(char *input, char *args[])
{
    int argc = 0;
    int i = 0;

    while (input[i] != '\0' && argc < MAX_ARGS - 1)
    {
```

```
/* Skip spaces */
while (input[i] == ' ' || input[i] == '\t')
{
    i++;
}

if (input[i] == '\0')
{
    break;
}

/* Start of a new argument */
args[argc++] = &input[i];

int write_pos = i;

while (input[i] != '\0')
{
```

```
/* Backslash escape */
if (input[i] == '\\' && input[i + 1] != '\0')
{
   i++;
   input[write_pos++] = input[i++];
}

/* Normal character */
else if (input[i] != ' ' && input[i] != '\t')
{
   input[write_pos++] = input[i++];
}

/* Space ends the current argument */
else
{
   i++;
   break;
```

```
            }
        }

        /* Terminate argument */
        input[write_pos] = '\0';
    }

    /* exec requires NULL at the end */
    args[argc] = NULL;

    return argc;
}

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];
```

```
printf("=========================================\n");
printf("      OSSP PROGRAM 6\n");
printf(" Escape Sequences & Process Execution\n");
printf("=========================================\n");

printf("\nExamples:\n");
printf(" ls -l\n");
printf(" echo hello\n");
printf(" echo hello\\ world\n");
printf(" pwd\n");
printf(" date\n");

printf("\nType 'exit' to quit.\n\n");

while (1)
{
   printf("shell6> ");
   fflush(stdout);
```

```
/* Read command */
if (fgets(input, sizeof(input), stdin) == NULL)
{
   break;
}

/* Remove newline */
input[strcspn(input, "\n")] = '\0';

/* Ignore empty input */
if (strlen(input) == 0)
{
   continue;
}

/* Exit */
if (strcmp(input, "exit") == 0)
```

```
{
    break;
}

/* Parse command */
int argc = parse_command(input, args);

if (argc == 0)
{
    continue;
}

/* Display arguments */
printf("\nParsed arguments:\n");

for (int i = 0; i < argc; i++)
{
    printf(" argv[%d] = [%s]\n", i, args[i]);
```

```
}

/* Create child process */
pid_t pid = fork();

if (pid < 0)
{
   perror("fork");
   continue;
}

/* CHILD PROCESS */
if (pid == 0)
{
   printf("\nChild process created.\n");
   printf("Child PID: %d\n", getpid());

   /* Execute command */
```

```
    execvp(args[0], args);

    /* Reached only if execvp fails */
    perror("execvp");
    exit(EXIT_FAILURE);
}

/* PARENT PROCESS */
else
{
    printf("Parent PID: %d\n", getpid());
    printf("Child PID: %d\n", pid);

    printf("Parent waiting for child...\n");

    int status;

    /* Wait for child to finish */
```

```
        waitpid(pid, &status, 0);

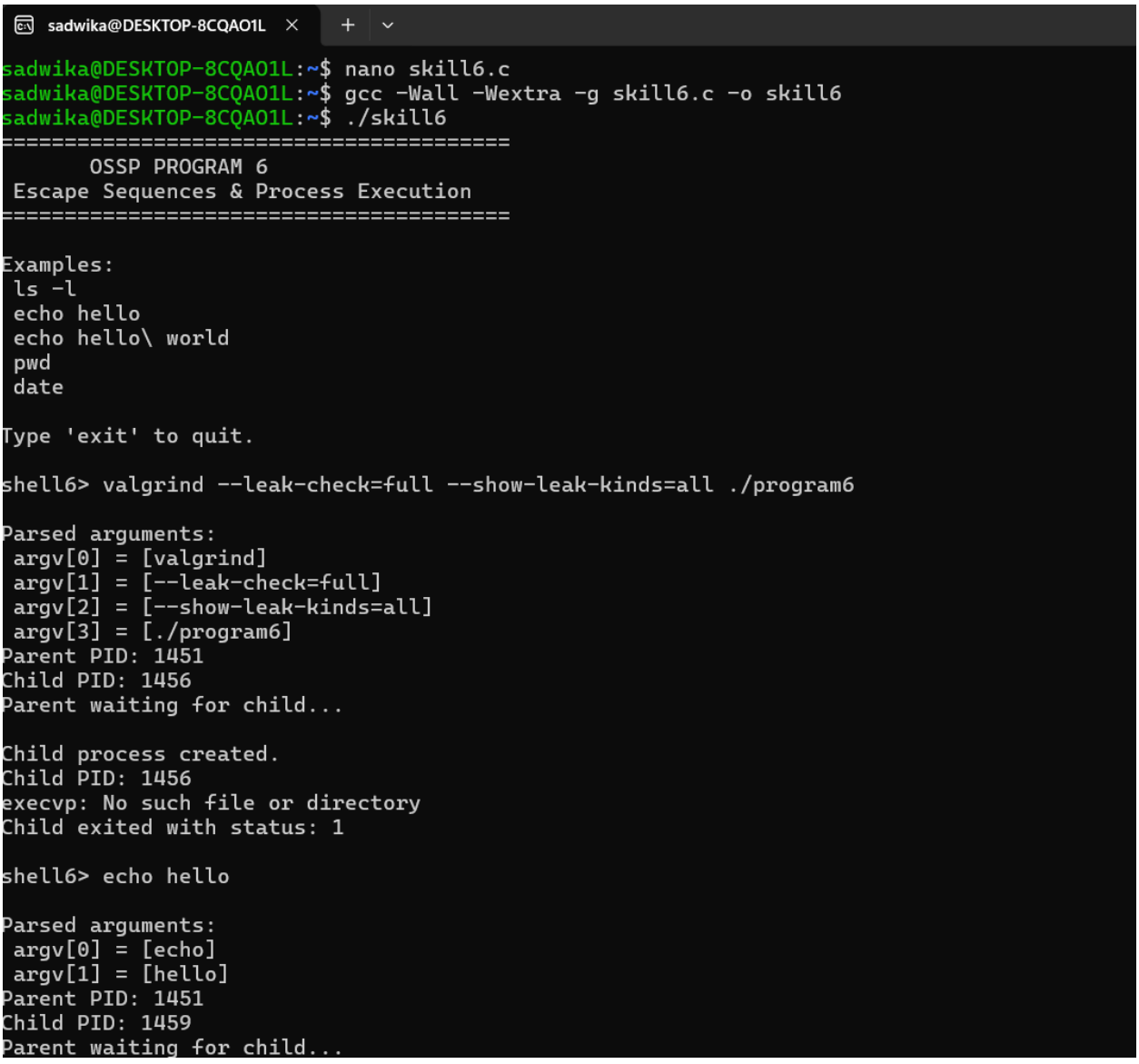
        /* Check child status */
        if (WIFEXITED(status))
        {
            printf(
                "Child exited with status: %d\n",
                WEXITSTATUS(status)
            );
        }
        else
        {
            printf("Child terminated abnormally.\n");
        }
    }

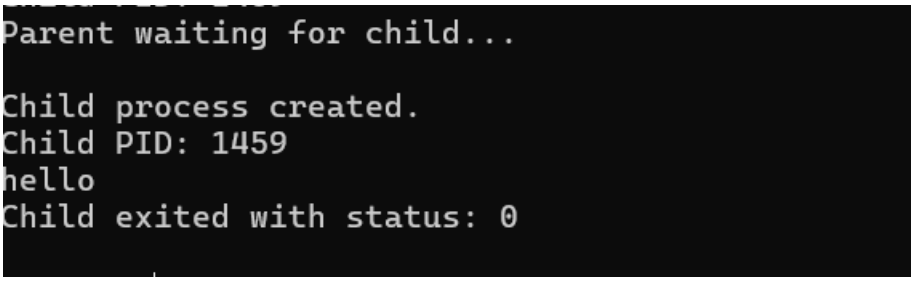
    printf("\n");
}
```

```
    printf("Program 6 terminated.\n");


    return 0;
}
```

OUTPUT :




Parent waiting for child...
Child process created.
Child PID: 1459
hello
Child exited with status: 0